

Chuyên đề quy hoạch động trên cây

Cố Dật Hoàn

16 tháng 8, 2016

Dynamic Programming / Tree DP selected topics - Gu Yihong

Abstract

Bản gốc là một bộ slide Beamer: 142 trang PDF chủ yếu đến từ các lớp phủ từng bước của khoảng 40 slide nội dung. Bản tiếng Việt này chuyển slide thành ghi chú đồng hành, giữ các trạng thái, công thức chuyển và ví dụ thuật toán chính; những hình minh họa đơn giản được diễn giải trong phần văn bản.

Contents

1	Cách nắm chắc quy hoạch động	2
2	Bài toán kinh điển: đường đi ngắn nhất lớn nhất	2
2.1	Trường hợp 1: đồ thị thông thường	2
2.2	Trường hợp 2: cây	2
2.2.1	Cách tham lam	2
2.2.2	Cách quy hoạch động	2
2.3	Mở rộng: <i>down</i> và <i>up</i>	3
2.4	Trường hợp 3: đồ thị một chu trình	3
2.5	Trường hợp 4: cactus	3
3	Kiến thức chuẩn bị	4
4	Ví dụ kinh điển	4
4.1	SPOJ MTREE	4
4.2	SPOJ GS	4
4.3	CodeChef TAPAIR	5
4.4	SPOJ TREECST	5
4.5	SPOJ TWOPATHS	5
4.6	Codeforces 202 Div1 E	5
4.7	NOI2012 Day2: Công viên lạc lối	5
5	Bài suy nghĩ	5
5.1	FoxConnection	5
5.2	InducedSubgraphs	5

1 Cách nắm chắc quy hoạch động

Cốt lõi của quy hoạch động là trạng thái. Một trạng thái gồm hai phần: cách biểu diễn và cách chuyển.

Năng lực quan trọng nhất khi làm quy hoạch động là phân loại trường hợp và quy nạp. Vì vậy nên nắm chắc các dạng cơ bản: quy hoạch động thông thường, quy hoạch động trên cây, quy hoạch động chữ số, quy hoạch động nén trạng thái và các kỹ thuật tối ưu hóa quy hoạch động. Phần còn lại đến từ luyện tập nhiều bài, chẳng hạn các bài trên Topcoder.

Trong tài liệu này dùng các ký hiệu sau:

- x : đỉnh hiện đang xét;
- c : một con của x ;
- f : cha của x ;
- (x, y) : đường đi từ x đến y ;
- $w(x, y)$: độ dài đường đi từ x đến y .

2 Bài toán kinh điển: đường đi ngắn nhất lớn nhất

Bài toán mẫu xuyên suốt các slide là: cho một đồ thị vô hướng G , hãy tìm giá trị lớn nhất của khoảng cách ngắn nhất giữa hai đỉnh bất kỳ. Trên cây, đó chính là đường kính cây; trên đồ thị một chu trình hoặc cactus, cần xử lý thêm phần chu trình.

2.1 Trường hợp 1: đồ thị thông thường

Cách thô là liệt kê đỉnh xuất phát i , chạy SPFA hoặc một thuật toán đường đi ngắn nhất thích hợp, rồi lấy giá trị lớn nhất. Có thể dùng Floyd với độ phức tạp $O(n^3)$ nếu miền dữ liệu nhỏ và đồ thị đủ dày.

2.2 Trường hợp 2: cây

2.2.1 Cách tham lam

Chọn tùy ý một đỉnh x , tìm đỉnh y xa x nhất trong cây. Sau đó tìm đỉnh z xa y nhất. Đường đi từ y đến z là một đường kính của cây.

Ý tưởng chứng minh là phản chứng. Nếu tồn tại một đường đi (y', z') dài hơn (y, z) , thì các đỉnh y, z, y', z' đều là lá. Xét theo việc hai đường đi (y, z) và (y', z') có giao nhau hay không, luôn suy ra mâu thuẫn với cách chọn y hoặc z là đỉnh xa nhất.

2.2.2 Cách quy hoạch động

Gốc hóa cây tại đỉnh 1. Gọi $L[x]$ là độ dài đường đi dài nhất nằm hoàn toàn trong cây con gốc x . Khi đó $L[1]$ là đáp án.

Để tính $L[x]$, phân loại đường đi dài nhất trong cây con của x :

- đường đi không qua x : lấy giá trị lớn nhất trong các $L[c]$;
- đường đi qua x : dùng hai nhánh đi xuống tốt nhất từ các con của x .

Gọi $down[x]$ là độ dài đường đi dài nhất bắt đầu tại x và đi xuống trong cây con. Khi đó

$$down[x] = \max_c \{down[c] + w(x, c)\}.$$

Đường đi tốt nhất qua x là tổng của giá trị lớn nhất và lớn nhì trong các $down[c] + w(x, c)$, trong đó giá trị lớn nhì có thể khởi tạo bằng 0.

2.3 Mở rộng: $down$ và up

Bài toán mở rộng: G là cây, đồng thời cần biết với mỗi đỉnh x , đường đi dài nhất bắt đầu từ x là bao nhiêu.

Phân loại bước đầu tiên:

- đi xuống: đóng góp là $down[x]$;
- đi lên: gọi $up[x]$ là đường đi dài nhất bắt đầu tại x với bước đầu tiên đi lên cha.

Để tính $up[x]$, giả sử bước đầu tiên đi từ x lên f . Bước tiếp theo có hai khả năng:

- tiếp tục đi lên: dùng $up[f]$;
- đi xuống một nhánh khác của f : nếu $down[f]$ không được chuyển từ chính x , dùng $down[f]$; nếu không, dùng giá trị tốt thứ hai $down2[f]$.

Vì vậy

$$up[x] = w(x, f) + \max\{up[f], \text{best_down_from_f_excluding_x}\}.$$

Cần lưu thêm $down2[x]$, giá trị lớn nhì của $down[c] + w(x, c)$, và $down1v[x]$, đứa con tạo ra $down[x]$.

2.4 Trường hợp 3: đồ thị một chu trình

Giả sử G chỉ có đúng một chu trình. Cần chú ý rằng bài toán vẫn hỏi giá trị lớn nhất của khoảng cách ngắn nhất. Trước hết chạy DFS hoặc BFS để tìm chu trình, ghi các đỉnh trên chu trình là $cir[i]$.

Phân loại đường đi:

- không đi qua chu trình: xử lý như trên cây và lấy lớn nhất trong các $L[cir[i]]$;
- đi qua chu trình: chọn hai đỉnh x, y trên chu trình, cộng hai nhánh cây treo và đoạn ngắn hơn trên chu trình.

Cách thô liệt kê hai đỉnh trên chu trình:

$$down[x] + down[y] + \min\{w_1(x, y), w_2(x, y)\},$$

với w_1, w_2 là hai khoảng cách theo hai chiều trên chu trình. Độ phức tạp là $O(k^2)$, trong đó k là độ dài chu trình.

Cách tối ưu: cắt chu trình thành một dãy và nhân đôi dãy đó. Chọn một hướng cố định. Khi liệt kê đỉnh x , các đỉnh y mà đi theo hướng này đến x là đoạn ngắn hơn tạo thành một khoảng $[l_x, r_x]$. Hai biên l_x, r_x đều đơn điệu không giảm, nên dùng hàng đợi đơn điệu để duy trì cực trị. Độ phức tạp còn $O(k)$.

2.5 Trường hợp 4: cactus

Ở đây G là cactus, nghĩa là hai chu trình bất kỳ có nhiều nhất một đỉnh chung. Chạy DFS một lần để lấy thứ tự DFS. Với mỗi đỉnh x , xét từng cạnh (x, y) :

- nếu là cạnh cây, cập nhật như quy hoạch động trên cây;

- nếu không phải cạnh cây, cạnh này thuộc một chu trình cần xử lý riêng.

Trên mỗi chu trình, làm từ thứ tự DFS lớn về nhỏ. Thứ tự tính đáp án và cập nhật *down* của một chu trình liên quan đến đỉnh có thứ tự DFS nhỏ nhất trong chu trình đó. Các cạnh nằm trên chu trình không trực tiếp cập nhật *down*. Trong một chu trình, phần có thể ảnh hưởng đến đáp án phía sau chỉ là đỉnh có thứ tự DFS nhỏ nhất.

Đoạn mã lỗi trong slide được giữ nguyên về ý tưởng và tên biến:

```
for (int k = c[x]; ~k; k = nxt[k]) {
    int y = g[k];
    if (fa[y] == x && low[y] > idx[x]) {
        ans = max(ans, down[y] + 1 + down[x]);
        down[x] = max(down[x], down[y] + 1);
    }
    if (fa[y] != x && idx[x] < idx[y]) {
        N = 0;
        for (int i = y; i != fa[x]; i = fa[i])
            cir[++N] = i;
        updans();
        for (int i = 1; i < N; ++i)
            ckmax(down[x], down[cir[i]] + min(i, N - i));
    }
}
```

3 Kiến thức chuẩn bị

Các slide liệt kê một số nền tảng nên có trước khi học sâu phần này:

- tổ hợp và các cách khử trùng lặp đơn giản;
- gộp theo heuristic, thường là gộp nhỏ vào lớn;
- băm chuỗi;
- xác suất, biến ngẫu nhiên rời rạc và kỳ vọng.

4 Ví dụ kinh điển

4.1 SPOJ MTREE

Cho một cây, tính tổng tích trọng số cạnh trên mọi đường đi, lấy modulo. Giới hạn $N \leq 10^5$. Đây là dạng quy hoạch động cộng dồn đóng góp đường đi qua mỗi đỉnh hoặc mỗi cạnh.

4.2 SPOJ GS

Cho một cây. Tại mỗi đỉnh, xác suất đi sang từng đỉnh kề đã được cho trước. Hỏi kỳ vọng số bước đi từ một đỉnh đến một đỉnh khác. Giới hạn $N \leq 10^5$. Dạng này thường dẫn đến hệ phương trình hoặc hai lượt DP trên cây để biểu diễn kỳ vọng theo cha/con.

4.3 CodeChef TAPAIR

Cho một đồ thị vô hướng liên thông N đỉnh. Hỏi có bao nhiêu cách xóa hai cạnh để đồ thị trở nên không liên thông. Giới hạn $N, M \leq 10^5$. Bài liên quan đến cầu, cây DFS và việc đếm cặp cạnh theo cấu trúc khối.

4.4 SPOJ TREECST

Xóa một cạnh của cây rồi thêm một cạnh khác để vẫn được một cây, sao cho đường kính của cây mới nhỏ nhất; cần in cả phương án. Giới hạn $N \leq 3 \cdot 10^5$. Các đại lượng đường kính, tâm cây và giá trị *down/up* thường là phần lõi.

4.5 SPOJ TWOPATHS

Trên một cây, tìm hai đường đi không giao nhau nghiêm ngặt sao cho tích độ dài của chúng là lớn nhất, rồi in tích đó. Giới hạn $N \leq 10^5$. Cần kết hợp đường kính trong cây con với thông tin tốt nhất bên ngoài hoặc theo cạnh cắt.

4.6 Codeforces 202 Div1 E

Cho một cây n đỉnh, trong đó có m đỉnh là tu viện. Mỗi tu viện lập danh sách các tu viện xa nó nhất và chuẩn bị đi thăm tất cả các nơi đó. Một đỉnh không có tu viện có thể bị xóa; nếu điều đó khiến một tu viện không còn nơi nào để đi thăm, giá trị vui thích tăng thêm 1. Hỏi giá trị vui thích lớn nhất và số phương án đạt giá trị đó. Giới hạn $m \leq n \leq 10^5$.

4.7 NOI2012 Day2: Công viên lạc lối

Cho đồ thị vô hướng liên thông n đỉnh, m cạnh. Ban đầu đứng ngẫu nhiên ở một đỉnh; mỗi bước chọn đều trong các đỉnh kề chưa đi qua. Hỏi kỳ vọng số bước di chuyển. Đề bảo đảm $m = n - 1$ hoặc $m = n$; nếu $m = n$, đồ thị có đúng một chu trình và chu trình có kích thước không quá 30, với $n \leq 10^5$.

5 Bài suy nghĩ

5.1 FoxConnection

Cho một cây n đỉnh; mỗi đỉnh có thể có một con fox. Cần di chuyển các con fox sao cho mỗi đỉnh có nhiều nhất một con, tập các đỉnh có fox liên thông, và tổng khoảng cách di chuyển là nhỏ nhất. Giới hạn $n \leq 50$.

5.2 InducedSubgraphs

Cho một cây n đỉnh. Cần đánh nhãn lại các đỉnh sao cho với mỗi tập liên tiếp

$$\{1, 2, \dots, k\}, \{2, 3, \dots, k+1\}, \dots, \{n-k+1, n-k+2, \dots, n\},$$

cây con cảm sinh bởi tập đó đều có đúng k đỉnh. Giới hạn $n \leq 50$.

Slide tách hai trường hợp $2k \leq n$ và $2k \geq n$. Với $2k \leq n$, cấu trúc bắt buộc là một chuỗi dài ở giữa, hai đầu treo các cây con kích thước k :

$$[STA] - (k+1) - (k+2) - \dots - (n-k) - [STB].$$

Ta liệt kê chuỗi giữa, tìm hai phần *STA* và *STB*. Việc tô màu trong *STA* tương đương yêu cầu mọi con đều có nhãn nhỏ hơn cha, từ đó đếm số cách tô màu.